Joint Optimization of Request Assignment and Computing Resource Allocation in Multi-Access Edge Computing

Haolin Liu, Xiaoling Long, Zhetao Li, *Member, IEEE*, Saiqin Long, Rong Ran, *Member, IEEE*, and Hui-Ming Wang, *Senior Member, IEEE*

Abstract—With the development of multi-access edge computing (MEC), the cloudlet at the edge of the network can provide nearby high-performance computing services, thus reducing the computational consumption of user equipments (UEs). To provide more real-time computing services to UEs, service providers face the challenge of optimizing the assignment of requests and the allocation of cloudlets' computing resources to achieve low latency while dealing with the large number of offloaded requests from UEs. Therefore, in this paper, we study the problem of minimizing the total latency to complete the requests in the MEC network by jointly optimizing request assignment and computing resource allocation. The problem is formulated as a mixed integer nonlinear programming (MINLP) problem which is NP-hard. To solve the problem, we decompose the problem into two subproblems which respectively optimize the request assignment and the computing resource allocation. We first deal with the computing resource allocation problem by utilizing the Lagrangian multiplier method, and the resulting solution is applied for the request assignment problem. Then a novel primal-dual based approximation algorithm is devised to address the request assignment problem. Finally, to verify the efficiency of the proposed algorithm, we provide an upper bound on the approximation ratio. The experiment results show that the proposed algorithm outperforms baseline algorithms in terms of total latency, loading balancing, and computational speed.

Index Terms—Approximation algorithm, computing resource allocation, joint optimization, latency minimization, multi-access edge computing, request assignment

1 Introduction

With the rapid development of deep learning in the fields of computer vision and natural language processing, user equipment (UE) becomes more and more intelligent, which can significantly change people's daily life. The intelligent services and applications (e.g., wearable virtual reality streaming, mobile social media, mobile games, in-vehicle systems, automated driving) need many computing resources and result in massive amounts of data [1]. Due to the limited computational capability, storage space, and battery capacity of UEs, computation-intensive requests can be migrated to cloud centers in order to reduce energy consumption and prolong the lifetimes of UEs as well. However, while cloud computing services can help overcome the limitations of UEs, cloud centers are geographically distant from UEs and the transmission latency between the cloud and UEs cannot be neglected, which is intolerable for some latency-sensitive services and applications such as facial recognition, interactive gaming, and automated driving.

The concept of multi-access edge computing (MEC) has been proposed to address the above issues, which is considered as a key technology for future wireless networks [2]. Relaying the development of 5 G and other next generation mobile communication technologies, MEC extends cloud computing services to the edge by using cloudlets (servers or server clusters) co-located with access points (APs) near users, providing distributed, low-latency, and reliable computing services [3]. Intelligent services and network function virtualization (NFV) rely heavily on complex software libraries, the dependencies and operating environments of these software libraries should be carefully considered. However, edge cloudlets consist of many clusters of servers with varying performance and architectures, which poses a challenge for the deployment of software libraries.

Virtualization technologies, whether in the form of virtual machines (VMs) or containers, can provide some features, such as platform independence, resource abstraction, and isolation, to shield requests for intelligent services and virtualized network functions (VNFs) from the differences

- *Haolin Liu, Xiaoling Long, Zhetao Li, and Saiqin Long are with the School of Computer Science, Key Laboratory of Hunan Province for Internet of Things and Information Security, and the Hunan International Scientific and Technological Cooperation Base of Intelligent Network, Xiangtan University, Xiangtan 411105, China. E-mail: liu.haolin@foxmail.com, longxiaolingll@163.com, liztchina@hotmail.com, saiqinlong@xtu.edu.cn.*
- *Rong Ran is with the Department of Electrical and Computer Engineering, Ajou University, Suwon 16499, South Korea. E-mail: sunnyran@ajou.ac.kr.*
- *Hui-Ming Wang is with the School of Information and Communications Engineering, Xi'an Jiaotong University, Xi'An, Shaanxi 710049, China, and also with the Ministry of Education Key Laboratory for Intelligent Networks and Network Security, Xi'an 710049, China. E-mail: xjbswhm@gmail.com.*

Manuscript received 3 Dec. 2021; revised 29 May 2022; accepted 31 May 2022. Date of publication 6 June 2022; date of current version 10 Apr. 2023.
This work was supported in part by the National Natural Science Foundation of China under Grants 61902336, 62032020, 62172350, 62172349, and 62171364, in part by the National Key Research and Development Program of China under Grant 2021YFB3101201, and in part by the Natural Science Foundation of Hunan under Grant 2021JJ40544.
(Corresponding author: Zhetao Li.)
Digital Object Identifier no. 10.1109/TSC.2022.3180105

1939-1374 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 01:37:41 UTC from IEEE Xplore. Restrictions apply.

between hardware and software environments in MEC [4]. This not only allows flexibility in adapting services to meet rapidly changing UEs but also reduces the need for frequent deployment and maintenance of physical infrastructure, thereby reducing costs [5]. Despite the benefits that the virtualization technology can bring, the latency-sensitive feature of the offloaded requests from UEs can still pose new challenges to the assignment of requests. More specifically, the capacity of computing resources allocated to virtualized instances in the cloudlet is limited, whereas there are multiple requests running on each instance, all of which will demand a certain proportion of computing resources [6]. The higher the proportion of computing resources obtained, the faster the request will be processed, and vice versa. There is also a corresponding boot time for both VMs and containers during the instance deployment [7]. In addition, the transmission time of the request needs to be considered. All these factors mentioned above affect the real-time performance of requests. Therefore, it is necessary to investigate a joint optimization of request assignment and computing resource allocation for service providers to reduce the completion latency of the requests from UEs.

In this paper, a problem of joint optimization of request assignment and computing resource allocation is studied with the aim of minimizing the total latency for completing the requests offloaded from UEs. We propose a novel model in terms of computing resource allocation, request assignment, and VM instance deployment for calculating the total latency. To solve the problem, we decompose it into a convex optimization problem for the computing resource allocation and an integer programming problem for the request assignment. We solve the subproblems by applying the Lagrangian multiplier method and devising an approximation algorithm, respectively. Our main contributions are summarized as follows.

- We formally define the joint optimization problem of the request assignment and the computing resource allocation, formulated it as a mixed integer nonlinear programming (MINLP) problem that minimizes the total latency to complete the requests, and prove it NP-hardness. The total latency is calculated by accumulating the booting times of deploying instances, the transmission latencies of requests, and the execution latencies for processing requests in the VM instances.
- The problem can be solved by decomposing it into two subproblems, namely the computing resource allocation problem and the request assignment problem. We prove that the computing resource allocation problem is convex and can be solved by using the Lagrangian multiplier method, and a novel approximation algorithm based on the primal-dual method is devised to solve the request assignment problem. Moreover, the time complexity and an upper bound on the approximation ratio of the proposed approximation algorithm is analyzed.
- The performance of the proposed algorithm is verified by devising experimental simulations. The proposed algorithm outperforms the random algorithm, the greedy algorithm, and the local search algorithm in terms of latency, load balancing, and computational speed.

The rest of this paper is organized as follows. Section 2 briefly reviews some related references. In Section 3, we introduce the system models and problem definition. An approximation algorithm is designed in Section 4. In Section 5, we perform the analysis of the algorithm and prove its approximation ratio and time complexity. In Section 6, we perform some simulation experiments to compare the proposed algorithm with the random algorithm, the greedy algorithm, and the local search algorithm. Finally, Section 7 concludes the paper in full.

2 Related Work

In recent years, MEC has drawn tremendous attention from researchers. In particular, task offloading has been extensively studied, mainly focusing on latency, energy consumption, and throughput in the offloading process. For example, Kuang *et al.* [8] formulated a partial offloading scheduling and resource allocation (POSP) problem in a single-user MEC system. To achieve the goal of minimizing the weighted sum of execution latency and energy consumption, a two-level alternation method framework based on Lagrangian dual decomposition was proposed. Alameddine *et al.* [3] formulated a dynamic task offloading and scheduling problem and solved it by logic-based Benders Decomposition. Ye *et al.* [9] considered three different offloading modes, namely partial offloading, complete local computation, and complete offloading, and then proposed a hybrid offloading scheme to operate them. Tian *et al.* [10] constructed a three-tier heterogeneous service system of multiple UEs, cloudlets, and a single cloud. Then they proposed a response ratio offloading strategy (RROS) algorithm to solve the offloading optimization problem. Liang *et al.* [11] investigated a framework of joint migration-and-handover by considering the user mobility for multi-user multi-cell MEC systems.The objectives are maximizing the sum offloading rate and minimizing the migration cost. Tran *et al.* [12] studied a joint optimization problem of task offloading and resource allocation in MEC and constructed a utility maximization problem with task latency and energy consumption as factors. Liao *et al.* [13] proposed an optimization problem model considering offloading decisions, communication power and computing resource allocation, and solved it using a heuristic method. Plachy *et al.* [14] studied a problem of optimal allocation of computing and communication resources in task offloading for mobile users, where computing resource optimization involves the migration of VMs and communication resource optimization involves the mobility of users.

Because of the limited computing resource capacity of edge servers or cloudlets, several studies have also focused on the resource allocation problem in MEC, including VM placement and service function chain (SFC) placement. For example, Rodrigues *et al.* [15] proposed an algorithm that utilizes VM migration and transmission power control to balance the workload between cloudlets to maximize cost-effectiveness. Zhao *et al.* [5] studied the optimal placement of VM copies to minimize the mean response time under various request demands among multiple applications and

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 01:37:41 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Comparison of Existing Works

	Offloading decision	Task/ Request assignment	VM deployment cost	Computing resource allocation	Objective	Methods
Kuang *et al.* [8]	Yes	No	No	No	Minimize the weighted sum of execution latency and energy consumption	An alternation method based on Lagrangian dual decomposition
Alameddine *et al.* [3]	No	Yes	No	Yes	Maximize the number of admitted tasks	A logic-based Benders decomposition method
Ye *et al.* [9]	Yes	No	No	No	Maximize the successful computation probability of tasks	Three distance dependent NOMA offloading schemes
Tian *et al.* [10]	Yes	Yes	No	No	Maximize the number of tasks offloaded and UEs involved	Greedy strategy
Liang *et al.* [11]	Yes	Yes	No	No	Maximize the sum offloading rate while reduce the total cost	Relaxation-and-rounding method
Tran *et al.* [12]	Yes	Yes	No	Yes	Maximize offloading utility	Bisection method and heuristic algorithm
Liao *et al.* [13]	Yes	Yes	No	No	Maximize the number of offloaded tasks	Heuristic algorithm
Plachy *et al.* [14]	No	Yes	Yes	Yes	Minimize the offloading latency	Heuristic algorithm with mobility prediction
Rodrigues *et al.* [15]	No	Yes	No	No	Maximize system scalability	Heuristic algorithm based on Particle Swarm Optimization
Zhao *et al.* [5]	No	Yes	No	No	Minimize the average response time	Enumeration method and heuristic algorithm
Bahreini *et al.* [16]	No	Yes	No	No	Maximize the social welfare	Greedy algorithm and approximation algorithm
Li *et al.* [17]	No	Yes	No	No	Maximize task admission profits	Approximation algorithm
Liu *et al.* [18]	No	Yes	No	No	Maximize the total amount of reward	Deep reinforcement learning method
Wang *et al.* [19]	No	Yes	No	No	Maximize the resource utilization	Graph matching algorithm and Hungarian mapping method
This work	No	Yes	Yes	Yes	Minimize the total latency to complete the requests	Lagrangian multiplier method and approximation algorithm

capacity constraints of MEC servers. Bahreini *et al.* [16] proposed a resource allocation and pricing model to maximize social welfare (SW) for the VM placement problem in the cloud and edge, and the results are obtained by an auction algorithm and an approximation algorithm. Li *et al.* [17] studied a robust SFC placement problem for service providers under the uncertainty assumption of both computing resource and data rates demanded by request executions, and proposed an approximation algorithm for solving it. Liu *et al.* [18] designed a deep reinforcement learning optimization framework for dynamic SFC Orchestration in MEC-enabled IoT network to reduce end-to-end latency. Wang *et al.* [19] investigated an SFC placement problem in MEC, and solved the CPU, Memory, and bandwidth resource constraints during SFC placement by graph matching and SFC mapping.

In summary, as Table 1 shows, most of the existing works mainly concentrate on the offloading decision for the user side and the task assignment for the server side. However, for the scenario where task offloading has been determined, there is a relatively rare work that takes into account factors such as VM deployment consumption and computing resource allocation proportion during task/request assignment. For this reason, this paper focuses on the dynamic allocation of computing resources and balanced assignment of requests on cloudlets to achieve overall latency minimization by considering the VM deployment consumption, i.e., the booting time of VM instances.

3 Preliminaries

In this section, the system models and notations are described. We first introduce the network model and then specify the communication and computation models in detail. Finally, the optimization problem is defined precisely.

3.1 Network Model

An undirected graph $G = (V \cup C, E)$ is used to present the MEC network, where V denotes a set of access points (APs) at different locations within the MEC network, such as bus stations, hospitals, shopping malls, airports, restaurants. $C = \{c_1, c_2, \ldots, c_j, \ldots, c_n\}$ denotes a set of cloudlets located at the same location as some APs, each cloudlet c_j has an available computing resource capacity f_j for the VM instance, and E represents a set of links between cloudlets and APs. We assume that each cloudlet is connected to an AP or other cloudlets via high-speed optical cables. Each AP covers a certain area where UEs can access the MEC service wirelessly through the AP. If a UE is located in an overlapping area of multiple APs, the UE can connect to its closest AP or the AP with the strongest signal strength for offloading its request. Fig. 1 shows an illustration of the MEC network.

Most of intelligent services, including computer vision, speech recognition, natural language processing, are based on deep learning methods that require a lot of computational capabilities to process [20]. Therefore, instead of considering UEs to perform these services locally, we implement these services by launching corresponding VM instances in cloudlets through virtualization techniques and migrating UE requests to run in the corresponding instances to obtain these services with the help of task offloading [11]. For this purpose, we assume that there is a set of requests (i.e., tasks offloaded from UEs) as $R = \{r_1, r_2, \ldots, r_i, \ldots, r_m\}$ to be processed in the MEC network, where the request $r_i = (w_i, s_i, d_i)$, w_i denotes

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 01:37:41 UTC from IEEE Xplore. Restrictions apply.

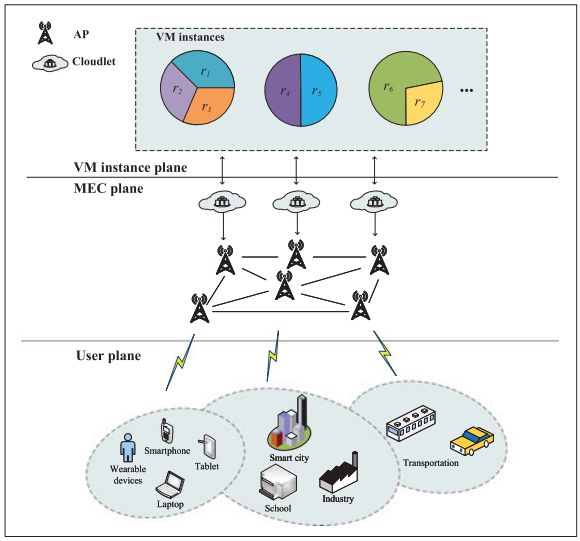


Fig. 1. An illustration of the MEC network.

the computation amount of request r_i, i.e., the total CPU cycles required to complete the request, s_i denotes the size of request r_i, i.e., the amount of data (e.g., data input and associated processing code) to be delivered to the cloudlet, d_i denotes the transmission distance of request r_i, where the transmission distance refers to the distance between request r_i and its nearest AP. We assume that each UE has only one computation-intensive request to be executed at a time, and the requests are not sequentially dependent and cannot be divided. In order to simplify the request model, the current requests are requesting a VM service with the same type. For the computing resource saving, it is assumed that the cloudlet will deploy the VM instance only when it is needed [16]. So if request $r_i \in R$ is assigned to be processed in a cloudlet without an instance deployment, the corresponding VM instance needs to be deployed in the cloudlet, and the boot time will be consumed. Since the VM instance can support multiple requests to be executed on it, the boot time is consumed at most once [21].

3.2 Transmission Model

The request completion latency includes the request transmission latency, which can have an impact on the request assignment. Referring to [22], [23], we employ OFDMA as the multi-access scheme for the MEC network. We assume that all UEs are working on orthogonal channels, where the spectrum of each AP connected to the UEs is divided into channels with equal bandwidth. Due to the non-overlapped sub-band, the intra-cell interference can be avoided. Then, the achievable data rate of the uplink from the UE with request r_i to its nearest AP can be computed by

$$\upsilon_i = Blog_2\left(1 + \frac{p_i g_i}{\sigma^2 + I_i}\right), \tag{1}$$

where B refers to the channel bandwidth, p_i denotes the transmission power at the UE where request r_i is located, g_i is the channel power gain. The background noise power is represented as σ^2. I_i is inter-cell interference power for the UE with request r_i, which needs to consider when there are other UEs connected to other APs and working on the same sub-band. Note that during the UE-AP association, the mobility of the UE is not high, thus g_i can be considered as a constant related to the distance d_i[22].

Since APs and cloudlets are linked by high-speed optical cables, which can reach a bandwidth of 4700 MHz.km [24], and the deployment area of MEC is not very large, we can regard the cable transmission latency between request r_i and cloudlet c_j as a fixed time and define it as T^c_{ij}.

In summary, the transmission latency between request r_i and cloudlet c_j can be expressed as

$$t^t_{ij} = \frac{s_i}{\upsilon_i} + T^c_{ij}. \tag{2}$$

3.3 Computation Model

We denote the computing resource capacity (CPU frequency) that the cloudlet can allocate to a particular VM instance as f_j. The execution latency of request r_i in the cloudlet as follows

$$t^e_{ij} = \frac{w_i}{\eta^j_i f_j}, \tag{3}$$

where η^j_i denotes the proportionality factor of computing resource capacity allocated by the VM instance in cloudlet c_j to request r_i. In order to enable requests to be executed in parallel, the computing resource capacity will be partitioned proportionally to each request that assigned on that VM instance [12].

3.4 Problem Formulation

In MEC, UEs' intelligent service requests are usually sensitive to latency. For providing satisfactory performance, the goal of our study is to ensure that the completion latency of offloaded requests is minimized through request assignment and computing resource allocation. Since the number of requests is fixed, minimizing the average latency and minimizing the total latency are equivalent, and to be convenient for description, this paper will aim at minimizing the total latency. The total latency for completing the requests includes the execution latencies, the transmission latencies, and boot times of VM instances in cloudlets. Specifically, this is achieved by selecting cloudlets to execute the requests and determining the proportional factors of computing resources allocated to the requests. We define an integer decision variable $x_{ij} \in \{0,1\}$, where $x_{ij} = 1$ if request r_i is assigned to cloudlet c_j, and $x_{ij} = 0$ otherwise. Define the integer decision variable $y_j \in \{0,1\}$, where $y_j = 1$ if the VM instance is deployed in cloudlet c_j, and $y_j = 0$ otherwise. Let $\boldsymbol{X} := \{x_{ij}\}_{r_i \in R, c_j \in C}$, $\boldsymbol{Y} := \{y_j\}_{c_j \in C}$, $\boldsymbol{\eta} := \{\eta^j_i\}_{r_i \in R, c_j \in C}$, the total latency minimization problem can be formally expressed as:

$$\text{P1}: \min_{\boldsymbol{X},\boldsymbol{Y},\boldsymbol{\eta}} \quad \sum_{j=1}^{n} y_j t^s_j + \sum_{i=1}^{m}\sum_{j=1}^{n} x_{ij} t^t_{ij} + \sum_{i=1}^{m}\sum_{j=1}^{n} x_{ij} t^e_{ij} \tag{4}$$

$$\text{s.t.} \quad x_{ij} \le y_j, \quad \forall r_i \in R, \forall c_j \in C, \tag{5}$$

$$\sum_{j=1}^{n} x_{ij} \ge 1, \quad \forall r_i \in R, \tag{6}$$

$$\sum_{i=1}^{m} x_{ij}\eta^j_i \le 1, \quad \forall c_j \in C, \tag{7}$$

$$x_{ij} \in \{0,1\}, \quad y_j \in \{0,1\}. \tag{8}$$

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 01:37:41 UTC from IEEE Xplore. Restrictions apply.

TABLE 2
Notations and Definitions

Notation	Definition
R	The set of requests
C	The set of cloudlets
C'	The set of cloudlets with deployed VM instance
Q	The set of unassigned requests
o_j	The set of requests assigned to cloudlet c_j
m	The number of requests
n	The number of cloudlets
w_i	The computation amount of request r_i
s_i	The data size of request r_i
d_i	The transmission distance of request r_i
t_{ij}^t	The transmission latency of request r_i offload to cloudlet c_j
t_{ij}^e	The execution latency of request r_i in cloudlet c_j
$t_{ij}^{e^*}$	The optimal execution latency of request r_i in cloudlet c_j
t_j^s	The boot time of VM instance in cloudlet c_j
t_{ij}^{t+e}	The sum of t_{ij}^t and $t_{ij}^{e^*}$
f_j	The computing resource capacity of the VM instance in cloudlet c_j
x_{ij}	To indicate whether request r_i is assigned to cloudlet c_j or not
y_j	To indicate whether cloudlet c_j deploy VM instance or not
η_i^j	The proportionality factor of computing resource allocated by cloudlet c_j to request r_i
η_i^*	The optimal proportionality factor of computing resource allocated to request r_i
α_i, β_{ij}	Dual variables

where t_j^s denotes the boot time of the VM instance in cloudlet c_j, which is assumed to be a constant in this paper. Constraint (5) denotes that cloudlet c_j must deploy a VM instance if a request is assigned to it. Note that a VM instance may be assigned multiple requests on it. Constraint (6) denotes that the request has to be assigned to at least one cloudlet. Since a request cannot be assigned multiple times and multiple assignments are not beneficial to the objective function, Constraint (6) can also be interpreted as a request having to be assigned to a certain cloudlet. Constraint (7) indicates that the computing resources allocated to the requests in cloudlet c_j should not exceed its total computing resource capacity. Table 2 shows the symbols in this paper.

3.5 NP-Hard Proof

Theorem 1. *P1 is an NP-hard problem.*

Proof. A special case of P1 is considered. Without considering the allocation proportionality factor of computing resources η_i^j, t_j^s in P1 can be considered as the opening cost for each facility, t_{ij}^t and t_{ij}^e as the connection cost of the facility and city, thus P1 can be transformed into a cost optimization problem which is the classical model of the facility location problem. The facility location problem aims to deploy some facilities and connect cities to the deployed facilities in such a way that the sum of the opening and connection costs is minimized. It is well known that the facility location problem is an NP-hard problem [25], so it follows that the more complex P1 is also an NP-hard problem. □

4 Algorithmic Solution

In this section, we deal with the latency minimization problem by utilizing the Lagrangian multiplier method and devising an approximation algorithm.

4.1 Problem Decomposition

The objective function of P1 can be reformulated as follows:

$$F(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{\eta}) = \sum_{j=1}^{n} y_j t_j^s + \sum_{i=1}^{m}\sum_{j=1}^{n} x_{ij} t_{ij}^t + \sum_{i=1}^{m}\sum_{j=1}^{n} x_{ij}\left(\frac{w_i}{\eta_i^j f_j}\right), \tag{9}$$

It can be seen that (9) is a MINLP problem with high complexity, where $\boldsymbol{X},\boldsymbol{Y}$ are the sets of binary variables and $\boldsymbol{\eta}$ is the set of continuous variables. To solve P1, by leveraging the Tammer decomposition method [26], we can decompose the objective function and constraints into two subproblems with different types of decision variables, i.e., P1 can be rewritten as the following equivalent form:

$$\begin{aligned} &\min_{\boldsymbol{X},\boldsymbol{Y}} \quad (\min_{\boldsymbol{\eta}} \quad F(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{\eta})) \\ &\text{s.t.} \quad \text{Constraints} \quad (5),(6),(7),(8)\,. \end{aligned} \tag{10}$$

Therefore, solving P1 is equivalent to solving the following two subproblems:

1) *Request assignment problem:*

$$\begin{aligned} \text{P1.1}: &\min_{\boldsymbol{X},\boldsymbol{Y}} \quad F(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{\eta}^*) \\ &\text{s.t.} \quad \text{Constraints} \quad (5),(6),(8)\,. \end{aligned} \tag{11}$$

When the optimal resource allocation policy $\boldsymbol{\eta}^*$ is given, the original optimization problem (9) can be transformed into a 0-1 integer programming problem $F(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{\eta}^*)$ with respect to $\boldsymbol{X},\boldsymbol{Y}$.

2) *Computing resource allocation problem:*

$$\begin{aligned} \text{P1.2}: &F(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{\eta}^*) = \\ &\min_{\boldsymbol{\eta}} \; F(\boldsymbol{X}^0,\boldsymbol{Y}^0,\boldsymbol{\eta}) \\ &\text{s.t.} \quad \text{Constraints} \quad (7)\,. \end{aligned} \tag{12}$$

To get $F(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{\eta}^*)$, we can temporarily fix $\boldsymbol{X}^0 = \{x_{ij}^0\}$. According to the constraint (5), $\boldsymbol{Y}^0 = \{y_j^0\}$ can also be fixed. $F(\boldsymbol{X},\boldsymbol{Y},\boldsymbol{\eta}^*)$ can be transformed into a minimization problem $F(\boldsymbol{X}^0,\boldsymbol{Y}^0,\boldsymbol{\eta})$ with respect to $\boldsymbol{\eta}$ where $\boldsymbol{X}^0$ and $\boldsymbol{Y}^0$ are determined.

Note that the decomposition from the original problem P1 to the request assignment problem P1.1 and the computing resource allocation problem P1.2 does not change the optimality [26]. Next, we will present the solutions of P1.2 and P1.1 using the Lagrangian method and a novel approximation algorithm, respectively, and finally obtain the solution of the original problem.

4.2 Computing Resource Allocation Problem

From the objective function of P1, it can be shown that cloudlet c_j allocates computing resources to a VM instance only when it is determined that request r_i is offloaded to cloudlet c_j (i.e., $x_{ij} = 1$), and it is known from constraint (6) that $y_j = 1$ when $x_{ij} = 1$, then the objective function (12) can be converted to the following function:

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 01:37:41 UTC from IEEE Xplore. Restrictions apply.

$$F(\boldsymbol{X}^0, \boldsymbol{Y}^0, \boldsymbol{\eta}) = \sum_{c_j \in C'} t_j^s + \sum_{c_j \in C'} \sum_{r_i \in o_j} \left(t_{ij}^t + \frac{w_i}{\eta_i^j f_j} \right), \tag{13}$$

where C' denotes the set of cloudlets with deployed VM instance and o_j denotes the set of requests assigned to cloudlet c_j.

Lemma 1. *Given $\boldsymbol{X}^0$ and $\boldsymbol{Y}^0$, the optimization problem P1.2 is a convex optimization problem with respect to $\boldsymbol{\eta}$.*

Proof. Since the request assignment policy has been assumed to be determined in this subproblem, η_i^j can be reduced to η_i, which is used to denote the proportionality factor of computing resource allocated to request r_i. Thus, it is known that $F(\boldsymbol{X}^0, \boldsymbol{Y}^0, \boldsymbol{\eta})$ is a function with respect to $\{\eta_1, \eta_2, \ldots, \eta_i, \ldots, \eta_m\}$. Then we obtain its Hessian matrix by taking partial derivatives of $\boldsymbol{\eta}$ as follows:

$$\mathbf{H} = \begin{bmatrix} \frac{\partial^2 F}{\partial^2 \eta_1} & \cdots & \frac{\partial^2 F}{\partial \eta_1 \partial \eta_{i'}} & \cdots & \frac{\partial^2 F}{\partial \eta_1 \partial \eta_m} \\ \vdots & \ddots & \vdots & \iddots & \vdots \\ \frac{\partial^2 F}{\partial \eta_i \partial \eta_1} & \cdots & \frac{\partial^2 F}{\partial \eta_i \partial \eta_{i'}} & \cdots & \frac{\partial^2 F}{\partial \eta_i \partial \eta_m} \\ \vdots & \iddots & \vdots & \ddots & \vdots \\ \frac{\partial^2 F}{\partial \eta_m \partial \eta_1} & \cdots & \frac{\partial^2 F}{\partial \eta_m \partial \eta_{i'}} & \cdots & \frac{\partial^2 F}{\partial^2 \eta_m} \end{bmatrix},$$

where the elements are expressed as

$$\frac{\partial^2 F}{\partial \eta_i \partial \eta_{i'}} = \begin{cases} \frac{2w_i}{(\eta_i)^3 f_j}, & \text{if} \quad i = i,' \\ 0, & \text{otherwise.} \end{cases} \tag{14}$$

From this we can obtain $\partial^2 F / \partial \eta_i \partial \eta_{i'} \geq 0$. Thus, all the eigenvalues of Hessian matrix $\mathbf{H}$ can be deduced as non-negative. Then Hessian matrix $\mathbf{H}$ is a symmetric semi-positive definite matrix [6]. As a result, it can be concluded that $F(\boldsymbol{X}^0, \boldsymbol{Y}^0, \boldsymbol{\eta})$ is a convex function. Since the constraints are linear, the computing resource allocation problem is a convex optimization problem. □

According to Lemma 1, following the Karush-Kuhn-Tucker (KKT) conditions [27], we can obtain Theorem 2.

Theorem 2. *Given $\boldsymbol{X}^0$ and $\boldsymbol{Y}^0$, the optimal solution of the objective function (13) with respect to $\boldsymbol{\eta}$ can be obtained as $\eta_i^* = \sqrt{w_i} \Big/ \sum_{r_i \in o_j} \sqrt{w_i}$.*

Proof. According to Lemma 1, the Lagrangian function with respect to (13) can be derived as follows:

$$\begin{aligned} L(\boldsymbol{\eta}, \boldsymbol{\lambda}) = & \sum_{c_j \in C'} t_j^s + \sum_{c_j \in C'} \sum_{r_i \in o_j} \left(t_{ij}^t + \frac{w_i}{\eta_i^j f_j} \right) \\ & + \sum_{c_j \in C'} \lambda_j (\sum_{r_i \in o_j} \eta_i^j - 1), \end{aligned} \tag{15}$$

where λ_j denotes the Lagrangian multiplier for constraint (7).

Assuming that $\bar{\boldsymbol{\eta}}$ and $\bar{\boldsymbol{\lambda}}$ are feasible solutions of $L(\boldsymbol{\eta}, \boldsymbol{\lambda})$. Based on the KKT conditions, for all $r_i \in R$, we can conclude that:

$$\begin{aligned} & \nabla F(\bar{\eta}_i^j) + \sum_{c_j \in C'} \bar{\lambda}_j \nabla \left(\sum_{r_i \in o_j} \bar{\eta}_i^j - 1 \right) = 0, \\ & \sum_{r_i \in o_j} \bar{\eta}_i^j - 1 = 0, \end{aligned} \tag{16}$$

where $\nabla F(\bar{\eta}_i^j)$ and $\nabla(\sum_{r_i \in o_j} \bar{\eta}_i^j - 1)$ is the first-order partial derivative of $F(\cdot)$ and $(\sum_{r_i \in o_j} \bar{\eta}_i^j - 1)$ with respect to $\bar{\eta}_i^j$, respectively. From (16) we can solve for $\bar{\eta}_i^j$, and get the optimal computing resource proportionality factor $\eta_i^* = \bar{\eta}_i^j$.

$$\eta_i^* = \frac{\sqrt{w_i}}{\sum_{r_i \in o_j} \sqrt{w_i}}. \tag{17}$$

□

Substituting (17) into (13), the optimal value of $F(\boldsymbol{X}^0, \boldsymbol{Y}^0, \boldsymbol{\eta})$ can be obtained as follows:

$$\begin{aligned} & F(\boldsymbol{X}^0, \boldsymbol{Y}^0, \boldsymbol{\eta}^*) \\ & = \sum_{c_j \in C'} t_j^s + \sum_{c_j \in C'} \sum_{r_i \in o_j} \left[\frac{\sqrt{w_i} \sum_{r_i \in o_j} \sqrt{w_i}}{f_j} + t_{ij}^t \right] \\ & = \sum_{c_j \in C'} t_j^s + \sum_{c_j \in C'} \left[\frac{(\sum_{r_i \in o_j} \sqrt{w_i})^2}{f_j} + \sum_{r_i \in o_j} t_{ij}^t \right]. \end{aligned} \tag{18}$$

4.3 Request Assignment Problem

As discussed above, given $\boldsymbol{X}^0$ and $\boldsymbol{Y}^0$, the optimal solution (18) is obtained. Through this, P1 can be transformed into an integer programming problem P1.1 with respect to $\boldsymbol{X}$ and $\boldsymbol{Y}$, Where the objective function (11) is expressed as

$$\begin{aligned} & F(\boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{\eta}^*) \\ & = \sum_{j=1}^{n} y_j t_j^s + \sum_{i=1}^{m} \sum_{j=1}^{n} x_{ij} t_{ij}^t + \sum_{i=1}^{m} \sum_{j=1}^{n} x_{ij} \frac{w_i}{\eta_i^* f_j} \end{aligned} \tag{19}$$

For ease of description, we simplify P1.1 as follows.

$$\text{P2}: \min_{\boldsymbol{X}, \boldsymbol{Y}} \quad \sum_{j=1}^{n} y_j t_j^s + \sum_{i=1}^{m} \sum_{j=1}^{n} x_{ij} (t_{ij}^t + t_{ij}^{e^*}) \tag{20}$$

$$\text{s.t.} \quad \text{Constraints} \quad (5), (6), (8),$$

where $t_{ij}^{e^*} = w_i / (\eta_i^* f_j)$.

To solve P2, the linear programming problem P3 is obtained first by relaxing the constraint (8) in P2, and then we work out the dual problem P4 of P3 and solve P4 by an approximation algorithm of primal-dual method. Specifically, P3 is the relaxed linear programming form of P2, which is expressed as follows:

$$\text{P3}: \min_{\boldsymbol{X}, \boldsymbol{Y}} \quad \sum_{j=1}^{n} y_j t_j^s + \sum_{i=1}^{m} \sum_{j=1}^{n} x_{ij} t_{ij}^{t+e} \tag{21}$$

$$\text{s.t.} \quad x_{ij}, y_j \geq 0, \quad \forall r_i \in R, \forall c_j \in C, \tag{22}$$

$$\text{Constraints} \quad (5), (6).$$

For simplicity, we denote $t_{ij}^t + t_{ij}^{e^*}$ by t_{ij}^{t+e}. Note that any feasible solution of P2 is also a solution of P3. We introduce

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 01:37:41 UTC from IEEE Xplore. Restrictions apply.

the dual variables α_i and β_{ij}, then transform P3 into a dual problem P4, which is formulated as follows.

$$\text{P4}: \max_{\boldsymbol{\alpha},\boldsymbol{\beta}} \quad \sum_{r_i \in R} \alpha_i \tag{23}$$

$$\text{s.t.} \quad \alpha_i - \beta_{ij} \le t_{ij}^{t+e}, \quad \forall r_i \in R, \forall c_j \in C\,, \tag{24}$$

$$\sum_{r_i \in R} \beta_{ij} \le t_j^s, \quad \forall c_j \in C\,, \tag{25}$$

$$\alpha_i, \beta_{ij} \ge 0, \quad \forall r_i \in R, \forall c_j \in C\,, \tag{26}$$

where $\boldsymbol{\alpha} = \{\alpha_i\}$ and $\boldsymbol{\beta} = \{\beta_{ij}\}$ are the sets of dual variables corresponding to the constraints (5) and (6) of P3, respectively.

We reformulate the constraint (24) in P4 to obtain

$$\alpha_i - t_{ij}^{t+e} \le \beta_{ij}\,. \tag{27}$$

Substituting (26) and (27) into (25) gives us

$$\sum_{r_i \in R} \max(\alpha_i - t_{ij}^{t+e}, 0) \le t_j^s\,. \tag{28}$$

In this case, only one dual variable set $\boldsymbol{\alpha}$ is left in P4, so P4 can be rewritten as:

$$\text{P4}: \max_{\boldsymbol{\alpha}} \quad \sum_{r_i \in R} \alpha_i$$
$$\text{s.t.} \quad \text{Constraint} \quad (28)\,.$$

According to the complementary slackness condition, the values of dual variables can be found to make the inequality of dual constraints to be tight. In this optimal case, the solution of the primal problem can then be obtained [28].

4.4 Approximation Algorithm

To address P4, we consider a primal-dual updating algorithm. Initially, let Q denote the set of unassigned requests, C indicates the set of cloudlets without a VM instance deployed, C' indicates the set of cloudlets that have been deployed the instance, and o_j denotes the set of requests assigned on c_j. We initialize the value of each dual variable to 0, i.e., $\alpha_i := 0$, and assume that all dual variables have the same incremental rate.

In the first step, we set a priority order of the requests assigned to each cloudlet. Specifically, the sum of the execution latency and transmission latency of the request r_i in cloudlet c_j is calculated first:

$$t_j(i) = \frac{w_i}{f_j} + t_{ij}^t\,. \tag{29}$$

The priority order of the requests $\Omega_j = \{r_{j_1}, r_{j_2}, \ldots, r_{j_m}\}$ in c_j is then sorted according to $t_j(i)$ in ascending order, i.e., $t_j(j_1) \le t_j(j_2) \le \cdots \le t_j(j_m)$, where j_1, j_2 and j_m are the 1st, 2nd, and mth request's index in Ω_j, respectively. *Procedure 1* shows the pseudo-code of this procedure.

Since constraint (28) holds for any $c_j \in C \cup C'$, the minimal dual value $\alpha_{\min}$ can be found by uniformly incrementing the values of all dual variables, such that there exists a cloudlet $c_j \in C \cup C'$ makes constraint (28) tight, i.e., the following equation is satisfied, which enables to obtain the values of primal variables.

$$\sum_{r_i \in Q} \max(\alpha_i - t_{ij}^{t+e}, 0) = t_j^s\,. \tag{30}$$

As α_i increases, there are two possible cases that need to be discussed.

Procedure 1. A Procedure for Generating the Priority Orders of the Requests

Input: request set R, cloudlet set C
Output: a sorted request set $\boldsymbol{\Omega}$
1: Initialization:$\boldsymbol{\Omega} \leftarrow \emptyset$
2: **for** each cloudlet c_j in C **do**
3: calculate $t_j(i) \leftarrow \frac{w_i}{f_j} + t_{ij}^t$ for each request $r_i \in Q$
4: get Ω_j by sorting Q in the ascending order by regarding the request's $t_j(i)$
5: $\boldsymbol{\Omega} \leftarrow \boldsymbol{\Omega} \cup \{\Omega_j\}$
6: **end for**
7: **return** $\boldsymbol{\Omega}$

Case I: Suppose that for any $c_j \in C$, there exist $o_j \subseteq Q$ and $r_i \in o_j$, such that $\alpha_i > t_{ij}^{t+e}$ and the following equation is satisfied:

$$\sum_{r_i \in o_j} \left(\alpha_i - t_{ij}^{t+e}\right) = t_j^s, \quad c_j \in C\,. \tag{31}$$

With this assumption, we can simplify (30) to (31). Case I helps us find the set o_j and the corresponding dual value α_i that first enables (31) to be tightened. At this point, we can believe that request $r_i \in o_j$ is contributing to the cloudlet deploying the VM instance that has not deployed one, so that it can deploy the instance in c_j and assign $r_i \in o_j$ to c_j. Since for any request r_i in the set Q, the initial values of the dual variables are 0 and the incremental rates are the identical, the values of α_i are the same. Thus, the dual value satisfying (31) can be calculated as:

$$\begin{aligned}
\alpha(j) &= \min_{\Omega_j^s \subseteq \Omega_j} \frac{t_j^s + \sum_{r_i \in \Omega_j^s} t_{ij}^{t+e}}{k} \\
&= \min_{\Omega_j^s \subseteq \Omega_j} \frac{t_j^s + \sum_{r_i \in \Omega_j^s} t_{ij}^t + \sum_{r_i \in \Omega_j^s} \frac{w_i}{\eta_i^s f_j}}{k} \\
&= \min_{\Omega_j^s \subseteq \Omega_j} \frac{t_j^s + \sum_{r_i \in \Omega_j^s} t_{ij}^t + \sum_{r_i \in \Omega_j^s} \left(\frac{w_i}{f_j} \times \frac{\sum_{r_i \in \Omega_j^s} \sqrt{w_i}}{\sqrt{w_i}}\right)}{k} \\
&= \min_{\Omega_j^s \subseteq \Omega_j} \frac{t_j^s + \sum_{r_i \in \Omega_j^s} t_{ij}^t + \left(\sum_{r_i \in \Omega_j^s} \sqrt{w_i}\right)^2 / f_j}{k}\,,
\end{aligned} \tag{32}$$

where $\alpha(j)$ is the minimal dual value that makes (31) hold, and $\Omega_j^s = \{r_{j_1}, r_{j_2}, \ldots, r_{j_k}\}$ is a subsequence of Ω_j that contains the first k requests in Ω_j, which is the set o_j we are looking for.

Case II: Suppose that for any $c_j \in C'$, there exists a request $r_{i'} \in Q$ that satisfies $\alpha_{i'} = t_{i'j}^{t+e}$, we can assign the request to c_j that has deployed the VM instance, and (31) holds too, i.e., (31) becomes as:

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 01:37:41 UTC from IEEE Xplore. Restrictions apply.

$$\sum_{r_i \in o_j \cup \{r_{i'}\}} \left(\alpha_i - t_{ij}^{t+e}\right) = t_j^s, \quad c_j \in C'. \tag{33}$$

Note that the new request $r_{i'}$ assigning to cloudlet c_j will calculate $\eta_{i'}^*$ first based on (17). Although t_{ij}^t remains unchanged, $t_{ij}^{e^*}$ of the assigned request r_i will be changed, which causes t_{ij}^{t+e} to be changed. Consequently, t_{ij}^{t+e} of the assigned request r_i in c_j will become as:

$$\begin{aligned} t_{ij}^{t+e} &= t_{ij}^t + t_{ij}^{e^*} = t_{ij}^t + w_i/(\eta_i^* f_j) \\ &= t_{ij}^t + w_i \Big/ \left(\frac{\sqrt{w_i}}{\sum_{i \in o_j} \sqrt{w_i} + \sqrt{w_{i'}}} \cdot f_j\right) \\ &= t_{ij}^t + \left(\sum\nolimits_{i \in o_j} \sqrt{w_i} + \sqrt{w_{i'}}\right) \sqrt{w_i} f_j \\ &(\forall c_j \in C,' \ \forall r_i \in o_j). \end{aligned} \tag{34}$$

The reassignment of the already assigned requests is not considered in our approximation algorithm. It poses a problem that as a new request is assigning to the cloudlet with the instance already deployed, t_{ij}^{t+e} of the requests satisfying (31) in Case I will become greater as (34) shows, and this will cause (31) to no longer hold. Therefore, in Case II, the dual value of request $r_{i'}$ must not only take into account the change in itself $t_{i'j}^{t+e}$, but also compensate for the change in t_{ij}^{t+e} of the already assigned request $r_i \in o_j$ due to the assignment of $r_{i'}$. We adopt the following approach to calculate the dual value $\alpha(j)$ in $c_j \in C'$ under Case II.

$$\begin{aligned} \alpha(j) &= \min_{r_{i'} \in \Omega_j} \left\{ t_{i'j}^{t+e} + t_j^s + \sum_{r_i \in o_j} t_{ij}^{t+e} - \sum_{r_i \in o_j} \alpha_i \right\} \\ &= \min_{r_{i'} \in \Omega_j} \left\{ t_j^s + \sum_{r_i \in o_j} t_{ij}^t + t_{i'j}^t + \sum_{r_i \in o_j} t_{ij}^{e^*} + t_{i'j}^{e^*} - \sum_{r_i \in o_j} \alpha_i \right\} \\ &= \min_{r_{i'} \in \Omega_j} \left\{ t_j^s + \sum_{r_i \in o_j \cup \{r_{i'}\}} t_{ij}^t + \sum_{r_i \in o_j \cup \{r_{i'}\}} \frac{w_i}{\eta_i^* f_j} - \sum_{r_i \in o_j} \alpha_i \right\} \\ &= \min_{r_{i'} \in \Omega_j} \left\{ t_j^s + \sum_{r_i \in o_j \cup \{r_{i'}\}} t_{ij}^t + \left(\sum_{r_i \in o_j \cup \{r_{i'}\}} \sqrt{w_i} \right)^2 / f_j - \sum_{r_i \in o_j} \alpha_i \right\}, \end{aligned} \tag{35}$$

where $r_{i'}$ denotes the request in Ω_j that minimizes (35).

Combining Case I and Case II, for $c_j \in C \cup C'$, suppose that c_{j^*} is the cloudlet where the minimum dual value can make (31) or (33) hold. So, we have

$$j^* = \arg\min_{c_{j^*} \in C \cup C'} \{\alpha(j^*)\}. \tag{36}$$

Then, we let $\alpha_{\min} := \alpha(j^*)$. Update the dual value of request r_i in the request set Q, i.e., let $\alpha_i := a_{\min}$, and keep the dual values of any r_i in Q the same. If $a_{\min}$ comes from Case I, we need to deploy the instance in c_{j^*}, i.e., $y_{j^*} := 1$, and then set $x_{ij^*} := 1$, where $r_i \in \Omega_{j^*}^s$. Otherwise, if $a_{\min}$ comes from Case II, it means that the VM instance has been deployed in c_{j^*}, and just assign $r_{i'}$ to c_{j^*}, i.e., $x_{i'j^*} := 1$. It is worth noting that we need to remove $\Omega_{j^*}^s$ or $r_{i'}$ from the set Q, thus ensuring that the requests already assigned do not continue to grow their dual values during the next iterations.

Since each assigned request is removed from Q, we can use Q being empty as the termination condition of the primal-dual updating algorithm.

Algorithm 1 shows the pseudo-code of our approximation algorithm for the total latency minimization problem. We calculate the optimal computing resource allocation proportions and invoke (32) and (35) corresponding to each of the two cases during the iterative process, and finally obtain the approximate optimal solution for the request assignment. Based on this result, we invoke (17) again to obtain the result of computing resource allocation.

For the request assignment and computing resource allocation with multiple service types of requests, Algorithm 1 can be run first for a set of requests of the same service type, and then the process can be repeated for the next service type of request set until all requests have been assigned.

Algorithm 1. An Approximation Algorithm for the Latency Minimization Problem

Input: request set R, cloudlet set C
Output: assignment result $\boldsymbol{X}, \boldsymbol{Y}$, computing resource allocation result $\boldsymbol{\eta}$

```
Initialization:
  Q ← R, C' ← ∅; x_ij ← 0, ∀r_i ∈ Q, ∀c_j ∈ C; y_j ← 0, ∀c_j ∈ C;
  α_i ← 0, ∀r_i ∈ Q; o_j ← ∅, ∀c_j ∈ C;
Get the sorted request set Ω by invoking Procedure 1;
while Q ≠ ∅ do
  α_min ← ∞
  for each cloudlet c_j ∈ C do
    if y_j ≠ 1 then
      Get α(j) and Ω_j^s by invoking (17) and (32);
      /*compute α(j) for the Case I*/
    else
      Select request r_i' to get α(j) by invoking (17) and (35);
      /*compute α(j) for the Case II*/
    end if
    if α(j) < α_min then
      α_min ← α(j); j* ← j
    end if
  end for
  α_i ← α_min for any r_i ∈ Q
  if c_j* ∈ C then
    Q ← Q\Ω_j*^s; o_j* ← Ω_j*^s; Ω_j* ← Ω_j*\Ω_j*^s;
    y_j* ← 1; x_ij* ← 1 for any r_i ∈ Ω_j*^s;
    C ← C\{c_j*}; C' ← C' ∪ {c_j*}
  else
    Q ← Q\{r_i'}; o_j* ← o_j* ∪ {r_i'}; Ω_j* ← Ω_j*\{r_i'};
    x_i'j* ← 1
  end if
end while
Caculate η by invoking (17) based on X, Y
return X, Y, η
```

5 Algorithm Analysis

In this section, we analyze the approximation ratio and time complexity of the proposed algorithm.

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 01:37:41 UTC from IEEE Xplore. Restrictions apply.

Lemma 2. *The proposed algorithm can produce a feasible primal solution.*

Proof. During the performance of the algorithm, the subset Ω^s_{j*} satisfying (31) found in Case I is eventually added to the assigned request set o_j and removed from the request set Q. Likewise, the request $r_{i'}$ satisfying (33) found in Case II is eventually added to o_j and removed from Q. The requests in o_j will not be reassigned and will stop growing their dual variables α_i. Therefore, there are no intersections between sets o_j, and a request can only be assigned to one cloudlet, which can avoid the violation of the primal constraints (5) and (8). The α_i of unassigned request will keep growing with the number of iterations for the algorithm. Even though t^{t+e}_{ij} can keep growing, there is an upper bound for it, and (30) is guaranteed to be satisfied, thus the algorithm is convergent. The algorithm will be iteratively executed until the request set Q is empty and the constraint (6) is satisfied. Therefore, we can conclude that the primal solution produced by the proposed algorithm does not violate the constraints, and it is a feasible solution. □

Lemma 3. *Suppose that the primal solution obtained by the proposed algorithm is $(\boldsymbol{X}, \boldsymbol{Y})$ and the dual solution is $\boldsymbol{a}$. The result of P2, P4 is defined as $P(\boldsymbol{X}, \boldsymbol{Y})$, $D(\boldsymbol{a})$, respectively. Then, $P(\boldsymbol{X}, \boldsymbol{Y}) = D(\boldsymbol{a})$.*

Proof. Both Case I and Case II are calculated according to (32) or (35). Therefore, for any cloudlet c_j that has deployed a VM instance, the following equation is satisfied at the end of the algorithm.

$$\sum_{r_i \in o_j} (\alpha_i - t^{t+e}_{ij}) = t^s_j, \quad \forall c_j \in C', \tag{37}$$

where α_i is the final dual value of the assigned request r_i. From which we can obtain the following equation as

$$\begin{aligned} D(\boldsymbol{a}) &= \sum_{c_j \in C'} \sum_{r_i \in o_j} \alpha_i = \sum_{c_j \in C'} (t^s_j + \sum_{r_i \in o_j} t^{t+e}_{ij}) \\ &= \sum_{c_j \in C'} t^s_j + \sum_{c_j \in C'} \sum_{r_i \in o_j} t^{t+e}_{ij} \\ &= \sum_{j=1}^{n} y_j t^s_j + \sum_{j=1}^{n} \sum_{i=1}^{m} x_{ij} t^{t+e}_{ij} \\ &= P(\boldsymbol{X}, \boldsymbol{Y}). \end{aligned} \tag{38}$$

□

In the following we will analyze the operation of the proposed algorithm in detail. For $c_j \in C'$, Case I is satisfied first, and then Case II, which corresponds to larger dual values. Thus, without loss of generality, for any $c_j \in C'$, we assume that $o^v_j = \{r_{\varepsilon_1}, r_{\varepsilon_2}, \ldots, r_{\varepsilon_i}, \ldots, r_{\varepsilon_v}\}$ is the set of requests assigned to c_j at the end of the algorithm, whose dual solutions' values $\{\alpha_{\varepsilon_1}, \alpha_{\varepsilon_2}, \cdots \alpha_{\varepsilon_i}, \ldots, \alpha_{\varepsilon_v}\}$ can satisfy the following order in such a way that

$$\alpha_{\varepsilon_1} = \alpha_{\varepsilon_2} = \cdots = \alpha_{\varepsilon_u} \leq \cdots \leq \alpha_{\varepsilon_v}, \tag{39}$$

where the dual values produced in Case I are $\{\alpha_{\varepsilon_1}, \alpha_{\varepsilon_2}, \ldots, \alpha_{\varepsilon_u}\}$ and the corresponding request set is $o^u_j = \{r_{\varepsilon_1}, r_{\varepsilon_2}, \ldots, r_{\varepsilon_u}\}$. The dual values produced in Case II is $\{\alpha_{\varepsilon_{u+1}}, \alpha_{\varepsilon_{u+2}}, \ldots, \alpha_{\varepsilon_v}\}$, and the corresponding request set is $o^v_j \backslash o^u_j = \{r_{\varepsilon_{u+1}}, r_{\varepsilon_{u+2}}, \ldots, r_{\varepsilon_v}\}$, which may be an empty set, i.e., the dual values are all produced in Case I.

Denote $t^{t+e}_{\varepsilon_i j}(o_j)$ as the sum of transmission and execution latency between request r_{ε_i} and cloudlet c_j when o_j is the current request set assigned to c_j. From (33), a new request added to o_j in Case II causes $t^{t+e}_{\varepsilon_i j}(o_j)$ of request r_{ε_i} in o^u_j to become greater, yet the dual value α_{ε_i} of r_{ε_i} has been fixed. When a certain number of requests are added to o_j, this may cause α_{ε_i} that has stopped growing to be smaller than the current $t^{t+e}_{\varepsilon_i j}(o_j)$. However, through (31) and (33), the following equation is satisfied when cloudlet c_j ends its request assignment.

$$\sum_{i=1}^{v} (\alpha_{\varepsilon_i} - t^{t+e}_{\varepsilon_i j}(o^v_j)) = t^s_j. \tag{40}$$

There may exist $\alpha_{\varepsilon_i} < t^{t+e}_{\varepsilon_i j}(o^v_j)$ for $r_{\varepsilon_i} \in o^u_j$ in (40), and thus it can be deduced that $\sum_{r_i \in o_j} \max(\alpha_i - t^{t+e}_{ij}, 0) > t^s_j$, which violates the dual constraint (28). It means that the dual solution produced by the proposed algorithm may not be a feasible dual solution.

To prove the performance of the proposed algorithm theoretically, we invoke the concept of approximation ratio. Given an optimization problem, the approximation ratio of an algorithm is defined as ξ, which satisfies $Approx/OPT \leq \xi$ when the problem is a minimization problem, where $Approx$ and OPT denote the approximate and optimal solution, respectively [28]. In the next, we prove the approximation ratio of the proposed algorithm by the dual fitting method [29].

Lemma 4. *$\boldsymbol{a}/\gamma$ is the feasible dual solution of P4 by dividing $\boldsymbol{a}$ by γ, where the upper bound of γ is 2.*

Proof. Without loss of generality, we consider the constraints on the request set o^v_j.

If request r_{ε_l} satisfies $u < l \leq v$. Based on (33) and (34), during the iteration of Case II, the following equation is satisfied for $u < l \leq v$.

$$\begin{aligned} t^s_j &= \sum_{i=1}^{l} \left(\alpha_{\varepsilon_i} - t^{t+e}_{\varepsilon_i j}(o^l_j)\right) \\ &= \sum_{i=1}^{l} \left(\alpha_{\varepsilon_i} - t^{t}_{\varepsilon_i j} - t^{e^*}_{\varepsilon_i j}(o^l_j)\right) \\ &= \sum_{i=1}^{l} \left(\alpha_{\varepsilon_i} - t^{t}_{\varepsilon_i j} - \frac{\sqrt{w_{\varepsilon_i}} \sum_{i=1}^{l} \sqrt{w_{\varepsilon_i}}}{f_j}\right) \\ &= \sum_{i=1}^{l-1} \left(\alpha_{\varepsilon_i} - t^{t+e}_{\varepsilon_i j}(o^{l-1}_j)\right) \\ &= \sum_{i=1}^{l-1} \left(\alpha_{\varepsilon_i} - t^{t}_{\varepsilon_i j} - t^{e^*}_{\varepsilon_i j}(o^{l-1}_j)\right) \\ &= \sum_{i=1}^{l-1} \left(\alpha_{\varepsilon_i} - t^{t}_{\varepsilon_i j} - \frac{\sqrt{w_{\varepsilon_i}} \sum_{i=1}^{l-1} \sqrt{w_{\varepsilon_i}}}{f_j}\right). \end{aligned} \tag{41}$$

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 01:37:41 UTC from IEEE Xplore. Restrictions apply.

Thus, we can have

$$\begin{aligned}
&\sum_{i=1}^{l}\left(\alpha_{\varepsilon_i} - t^t_{\varepsilon_i j} - \frac{\sqrt{w_{\varepsilon_i}}\sum_{i=1}^{l}\sqrt{w_{\varepsilon_i}}}{f_j}\right) \\
&\quad - \sum_{i=1}^{l-1}\left(\alpha_{\varepsilon_i} - t^t_{\varepsilon_i j} - \frac{\sqrt{w_{\varepsilon_i}}\sum_{i=1}^{l-1}\sqrt{w_{\varepsilon_i}}}{f_j}\right) \\
&= \alpha_{\varepsilon_l} - t^t_{\varepsilon_l j} - \frac{\left(\sum_{i=1}^{l}\sqrt{w_{\varepsilon_i}}\right)^2}{f_j} + \frac{\left(\sum_{i=1}^{l-1}\sqrt{w_{\varepsilon_i}}\right)^2}{f_j} \\
&= \alpha_{\varepsilon_l} - t^t_{\varepsilon_l j} \\
&\quad - \frac{\left(\sum_{i=1}^{l}\sqrt{w_{\varepsilon_i}} - \sum_{i=1}^{l-1}\sqrt{w_{\varepsilon_i}}\right)\left(\sum_{i=1}^{l}\sqrt{w_{\varepsilon_i}} + \sum_{i=1}^{l-1}\sqrt{w_{\varepsilon_i}}\right)}{f_j} \\
&= \alpha_{\varepsilon_l} - t^t_{\varepsilon_l j} - \frac{\sqrt{w_{\varepsilon_l}}\left(\sqrt{w_{\varepsilon_l}} + 2\sum_{i=1}^{l-1}\sqrt{w_{\varepsilon_i}}\right)}{f_j} = 0\,. \qquad (42)
\end{aligned}$$

Finally, we can get that

$$\alpha_{\varepsilon_l} = t^t_{\varepsilon_l j} + \frac{\sqrt{w_{\varepsilon_l}}\left(\sqrt{w_{\varepsilon_l}} + 2\sum_{i=1}^{l-1}\sqrt{w_{\varepsilon_i}}\right)}{f_j}\,, \qquad (43)$$

where $u < l \le v$.

Based on (43), we construct a function Γ on ε_l and γ as follows:

$$\begin{aligned}
\Gamma(\varepsilon_l,\gamma) &= \frac{\alpha_{\varepsilon_l}}{\gamma} - t^{t+e}_{\varepsilon_l j}(o^v_j) \\
&= \frac{t^t_{\varepsilon_l j}}{\gamma} + \frac{\sqrt{w_{\varepsilon_l}}\left(\sqrt{w_{\varepsilon_l}} + 2\sum_{i=1}^{l-1}\sqrt{w_{\varepsilon_i}}\right)}{\gamma f_j} \\
&\quad - t^t_{\varepsilon_l j} - \frac{\sqrt{w_{\varepsilon_l}}\sum_{i=1}^{v}\sqrt{w_{\varepsilon_i}}}{f_j} \\
&= \frac{(1-\gamma)t^t_{\varepsilon_l j}}{\gamma} \\
&\quad + \frac{w_{\varepsilon_l} + (2-\gamma)\sqrt{w_{\varepsilon_l}}\sum_{i=1}^{l-1}\sqrt{w_{\varepsilon_i}}}{\gamma f_j} - \frac{\sqrt{w_{\varepsilon_l}}\sum_{i=l}^{v}\sqrt{w_{\varepsilon_i}}}{f_j}\,. \qquad (44)
\end{aligned}$$

Substituting $\gamma = 2$ into the above function yields

$$\begin{aligned}
\Gamma(\varepsilon_l,2) &= \frac{(1-2)t^t_{\varepsilon_l j}}{2} \\
&\quad + \frac{w_{\varepsilon_l} + (2-2)\sqrt{w_{\varepsilon_l}}\sum_{i=1}^{l-1}\sqrt{w_{\varepsilon_i}}}{2f_j} - \frac{\sqrt{w_{\varepsilon_l}}\sum_{i=l}^{v}\sqrt{w_{\varepsilon_i}}}{f_j} \\
&= -\frac{t^t_{\varepsilon_l j}}{2} + \frac{w_{\varepsilon_l}}{2f_j} - \frac{\sqrt{w_{\varepsilon_l}}\sum_{i=l}^{v}\sqrt{w_{\varepsilon_i}}}{f_j} \\
&= -\frac{t^t_{\varepsilon_l j}}{2} + \frac{w_{\varepsilon_l}}{2f_j} - \frac{\sqrt{w_{\varepsilon_l}}\sqrt{w_{\varepsilon_l}}}{f_j} - \frac{\sqrt{w_{\varepsilon_l}}\sum_{i=l+1}^{v}\sqrt{w_{\varepsilon_i}}}{f_j} \\
&= -\frac{t^t_{\varepsilon_l j}}{2} - \frac{w_{\varepsilon_l}}{2f_j} - \frac{\sqrt{w_{\varepsilon_l}}\sum_{i=l+1}^{v}\sqrt{w_{\varepsilon_i}}}{f_j} \\
&\le 0\,. \qquad (45)
\end{aligned}$$

If request r_{ε_l} satisfies $l \le u$. From (31), we can get $\sum_{i=1}^{u}(\alpha_{\varepsilon_i} - t^{t+e}_{\varepsilon_i j}(o^u_j)) = t^s_j$. Since $t^{t+e}_{\varepsilon_i j}(o^u_j) \le t^{t+e}_{\varepsilon_i j}(o^v_j)$, it is able to derive that

$$\sum_{i=1}^{u}\max(\alpha_{\varepsilon_i}/\gamma - t^{t+e}_{\varepsilon_i j}(o^v_j), 0) \le t^s_j\,. \qquad (46)$$

Combining (45) and (46), the following relationship for any o^v_j and $\gamma = 2$ can be met

$$\begin{aligned}
&\sum_{i=1}^{v}\max(\alpha_{\varepsilon_i}/2 - t^{t+e}_{\varepsilon_i j}(o^v_j), 0) \\
&= \sum_{i=1}^{u}\max(\alpha_{\varepsilon_i}/2 - t^{t+e}_{\varepsilon_i j}(o^v_j), 0) + \\
&\qquad \sum_{i=u+1}^{v}\max(\Gamma(\varepsilon_i, 2), 0) \\
&\le t^s_j\,. \qquad (47)
\end{aligned}$$

In summary, there exists γ such that the dual constraint (28) holds, e.g.,

$$\sum_{r_i \in o_j}\max(\alpha_i/\gamma - t^{t+e}_{ij}, 0) \le t^s_j\,. \qquad (48)$$

Therefore, $\boldsymbol{a}/\gamma$ is a feasible solution. □

Theorem 3. *The approximation ratio of the proposed algorithm is γ and the time complexity is $o(nm^2)$.*

Proof. Suppose the optimal result of P2, P3, P4 is $OPT(\mathrm{P2})$, $OPT(\mathrm{P3})$, $OPT(\mathrm{P4})$, respectively, where $OPT(\mathrm{P2})$ is equal to the optimal result OPT of the total latency minimization problem. According to Lemmas 3 and 4, we can respectively get $P(\boldsymbol{X},\boldsymbol{Y}) = D(\boldsymbol{a})$, and $\boldsymbol{a}/\gamma$ is a feasible solution for P4. By the weak duality [28], the following inequality holds

$$\begin{aligned}
&D(\boldsymbol{a}/\gamma) \le OPT(\mathrm{P4}) \le OPT(\mathrm{P3}) \\
&\quad \le OPT(\mathrm{P2}) \le P(\boldsymbol{X},\boldsymbol{Y}) = D(\boldsymbol{a})\,. \qquad (49)
\end{aligned}$$

Thus, we can conclude that

$$P(\boldsymbol{X},\boldsymbol{Y}) \le \gamma \cdot OPT(\mathrm{P2}) = \gamma \cdot OPT\,, \qquad (50)$$

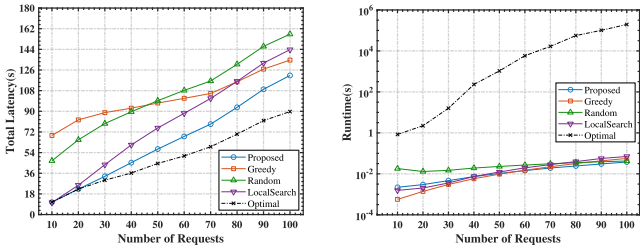
where $\gamma \le 2$. Therefore, the approximation ratio of the proposed algorithm is γ.

The time complexity of the proposed algorithm is analyzed as follows. It takes $O(nm\log m)$ time to execute Procedure 1. In the worst case, Case I takes $O(m)$ time to find Ω^s_j, and Case II also takes $O(m)$ time to find $r_{i'}$.The algorithm takes $O(nm^2)$ time to traverse the entire cloudlet and the set Q. Therefore, the total time required for the proposed algorithm is $O(nm\log m + nm^2)$. The time complexity is $O(nm^2)$. □

6 Performance Evaluation

The simulation experiments are run on Matlab R2019b hosted by a PC with an Intel i7 3.5 GHz CPU, and 16 GB RAM. The impact of important parameters on the performance of the proposed algorithm, including the number of requests, the number of cloudlets, the cloudlet's computing resource capacity, and the VM instance boot time, are mainly investigated.

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 01:37:41 UTC from IEEE Xplore. Restrictions apply.



(a) Total latency (b) Runtime

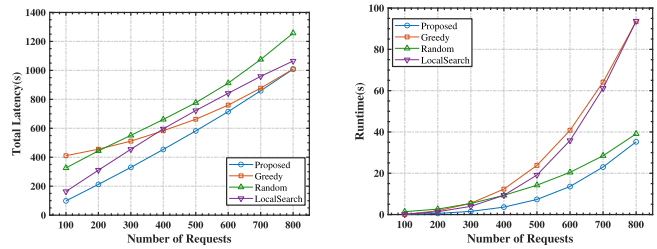
Fig. 2. Impact of the number of requests on algorithm performance in the small-scale network.

6.1 Experiment Setup

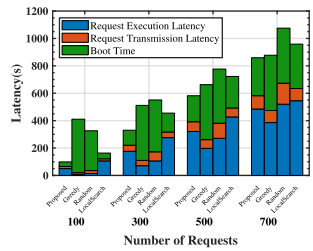
We consider a MEC network consisting of 100 APs, with some APs in the network that co-located with cloudlets, and the number of cloudlets n varies between 10 and 100 with an increment of 10. The number of requests m in the network is set from 10 to 800. According to [30], the cloudlet consists of several servers, where the computing resource capacity f_j of the VM instance in c_j can vary from 5 to 10 GHz. The boot time of the VM instance is related to the cloudlet settings and hardware, and varies on different cloudlets, so t_j^s is randomly drawn from $[4-12]$ seconds [31]. The data size s_i of request r_i is related to the transmission latency and is set to take values within $[1-3]$ MB. The computation amount of request w_i that are required by request r_i is related to the execution latency and is set randomly within $[200-400]$ Megacycles. The maximum transmission power $p_{\max}$ is set to 100 mW [32], and the transmission power p_i of the UE with request r_i does not exceed $p_{\max}$. Following [22], [23], the channel power gain between the AP and the UE with request r_i is modeled as $140.7 + 36.7log_{10}(d_i) + \mu$, where d_i denotes the transmission distance of request r_i, which is randomly set from $[0.02-0.4]$ km, and μ indicates the log-normal shadowing standard deviation, which follows $\mathcal{N}(0, 8\text{ dB})$. The cable transmission latency is randomly set within $[20-200]$ ms, and the channel bandwidth B is set to 2 MHz [33]. Unless otherwise noted, the parameters in the experiments will be adopted in the default setting as above. For each point, 100 independent runs are conducted in the experiments.

To evaluate the performance of the algorithm proposed in this paper, the algorithm is compared with the following three algorithms as

1) *Random:* The optimal fractional solution of the relaxed non-linear programming problem is calculated by ILOG CPLEX [34] and then randomly normalized to the integer solution 0 or 1 according to the probability associated with the values of the fractional solution.
2) *Greedy:* Ignore the latency for booting the VM instance, and only consider the transmission latency and execution latency of the request. In the process of the request assignment, each round, among the unassigned requests, select the request and the cloudlet with the minimum sum of transmission latencies and execution latencies, and then assign the request to that cloudlet. Repeat until all requests have been assigned. The allocation of computing resources for the request is obtained by (17).



(a) Total latency (b) Runtime



(c) Distribution of different latencies

Fig. 3. Impact of the number of requests on algorithm performance in the large-scale network.

3) *Local search:* First, the set of cloudlets that minimizes the total latency is found by adding, deleting, and swapping the cloudlets. Then, based on the selected cloudlet set, the greedy algorithm is used to find the request assignment solution. The local search algorithm can be considered as another joint optimization policy, optimizing the boot time first, then the execution latency and transmission latency.

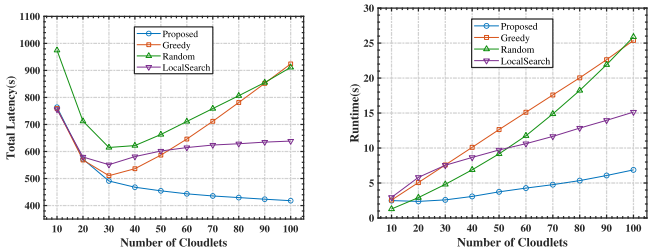
6.2 Impact of Number of Requests

The performance of the proposed algorithm against the greedy algorithm, the random algorithm, the local search algorithm, and the optimal solution *Optimal* is evaluated by varying the number of requests in a small-scale network. The solution of *Optimal* is obtained by using CPLEX to solve P1.

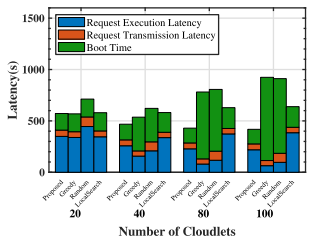
Fig. 2 shows the impact of the number of requests m on the algorithm performance, which ranges from 10 to 100 and the number of cloudlets n is fixed to 10. It is shown from Fig. 2a that the total latency obtained by the proposed algorithm is closest to the *Optimal*. In particular, when $m = 100$, the total latency of the proposed algorithm is only 1.35 times of the *Optimal*, while the other algorithms perform worse. In addition, we can see that the total latency for completing the requests increases as the number of requests increases, which is due to more requests being executed in cloudlets, thus increasing the total latency. Fig. 2b shows the runtime of the algorithms. Note that the y-axis is a log scale. When $m = 100$, the running time of the proposed algorithm is the shortest and is only 0.2 parts per million of the *Optimal*. Note that CPLEX requires a significant running time to obtain the optimal solution for the large-scale MINLP, so the *Optimal* is not considered in the large-scale network below.

We then investigate the impact of the large-scale network on the algorithms' performance by changing m ranges from 100 to 800 and fixing $n = 50$. As shown in Fig. 3a, the total latency increases with m increasing, and when $m \leq 700$, the proposed algorithm outperforms the other algorithms. For $m = 800$, the proposed algorithm is comparable to the

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 01:37:41 UTC from IEEE Xplore. Restrictions apply.

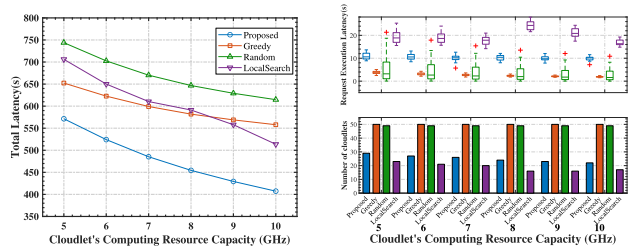


(a) Total latency (b) Runtime



(c) Distribution of different latencies

Fig. 4. Impact of the number of cloudlets on algorithm performance.



(a) Total latency (b) Number of occupied cloudlets and request execution latency

Fig. 5. Impact of the cloudlet's computing resource capacity on algorithm performance.

greedy algorithm. The reason is that the number of requests in the network is excessive at this time, and the total execution latency in cloudlets is larger than the latency consumed by booting the VM instances, which also means that the execution latency dominates the total latency, as shown in Fig. 3c. Then, to reduce the overall total latency, the best way is to deploy instances as much as possible to achieve load balancing. In this extreme case, just like the greedy algorithm, the proposed algorithm is also able to deploy as many instances as possible and optimize the execution and transmission latencies and thus minimizing the total latency, and also outperforms other algorithms significantly in terms of the running time which is shown in Fig. 3b. In Fig. 3c, we compare three latencies (the execution latency, the transmission latency, and the boot time of the VM instance) of the four algorithms with the different number of requests. As m grows, the higher the number of requests the less computing resources are allocated to each request, so more cloudlets are needed to balance the computational demand of the requests. It can be observed that the proposed algorithm has the lowest boot time at different scales, which means it occupies fewer cloudlets. We can also see that the proposed algorithm is weaker than the greedy algorithm in terms of both transmission latency and execution latency, but the greedy algorithm does not consider the boot time, resulting in the boot time greatly affecting the total latency. As seen in small-scale and large-scale scenarios, the proposed algorithm is highly scalable and can achieve advantages in terms of computational speed and total latency.

6.3 Impact of Number of Cloudlets

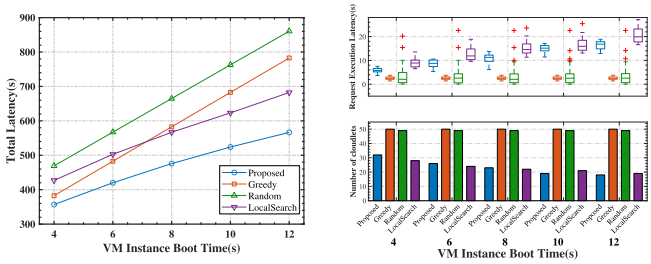
We then study the impact of the number of cloudlets on the performance of the proposed algorithm by varying n from 10 to 100 and fixing $m = 400$. It can be seen from Fig. 4a that the curve of the proposed algorithm always tends to decrease as n grows, which is because the proposed algorithm can effectively reduce the execution latency without occupying more cloudlets. The total latency curves of the other three algorithms tend to increase gradually because as n increases, more cloudlets are available to share the requests in the network, which reduces the execution latency to some extent, but also increases the number of cloudlets occupied, leading to a significant increase in the boot times of VM instances. This can also be verified from Fig. 4c, the boot time of the proposed algorithm is significantly lower than those of other algorithms.

As can be seen in Fig. 4b, the proposed algorithm is inferior to the random algorithm at $n = 10$, which is due to the faster solution speed of CPLEX when the problem size is relatively small. However, once n grows, the solution speed of CPLEX slows down and the advantage of the proposed algorithm in terms of computational speed is gradually shown. Overall, the advantages of the proposed algorithm are that it can obtain lower total latency at different cloudlet scales and occupy fewer cloudlets, which effectively improves the computing resource utilization.

6.4 Impact of Cloudlet's Computing Resource Capacity

In this section, we compare the performance of the proposed algorithm against the greedy algorithm, the random algorithm, and the local search algorithm, in terms of the cloudlet's computing resource capacity. The computing resource capacity of each cloudlet is assumed to follow a uniform distribution with a mean value from [5 6 7 8 9 10] GHz, and m is fixed to 400, n is fixed to 50, other parameters are set as the default values. In Fig. 5a, we can observe that the total latency of the proposed algorithm is better than the other three algorithms overall. The total latency is decreasing as the computing resource capacity increase, which is because the increase in resource capacity not only reduces the execution latency of the request, but also increases the number of requests that VM instance can host. Fig. 5b is a box-and-whiskers plot, for representing the distribution of request execution latencies on different cloudlets. In Fig. 5b, the box plot shows the distribution of execution latency of each algorithm at different computing resource capacities, and the bar chart shows the number of cloudlets occupied by different algorithms. As it shows, the proposed algorithm make a better compromise between cloudlet occupancy and execution latency, and achieves a more balanced distribution on execution latency except for the greedy algorithm. In addition, the proposed algorithm occupies significantly fewer cloudlets than the greedy algorithm and outperforms the total latency, which implies that the proposed algorithm can effectively improve resource utilization and thus achieve better load

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 01:37:41 UTC from IEEE Xplore. Restrictions apply.



(a) Total latency

(b) Number of occupied cloudlets and request execution latency

Fig. 6. Impact of the VM instance boot time on algorithm performance.

balancing. By the way, the reason why the greedy algorithm is more concentrated in the box plot is that it only considers how to assign the requests so that the sum of transmission and execution latencies is minimized. While balancing the load, the lack of consideration of the boot time of VM instance in the cloudlet leads to a sharp increase in the total latency as all cloudlets are occupied in this case.

6.5 Impact of Boot Time of VM Instance

In Fig. 6, the performance of the greedy algorithm, the random algorithm, the local search algorithm and the proposed algorithm are evaluated by varying the instance boot time, where it is assumed that the boot time t_j^s follows a uniform distribution with a mean value ranging from 4 to 12 seconds, and m is fixed to 400, n is fixed to 50. As shown in Fig. 6a, the overall total latency increases as the boot time grows. From the bar chart in Fig. 6b, it can be seen that the number of cloudlets occupied by the proposed algorithm decreases with the increase of the boot time, which is due to the need to reduce the boot times as much as possible for achieving the goal of the algorithm. As can be seen from the box plot in Fig. 6b, the execution latencies of the greedy and the random algorithm have no significant changes as the boot time increases, indicating that these two algorithms do not change their request assignment decisions based on the boot time. It is worth noting that although the request execution latency of the proposed algorithm is increasing, it still significantly outperforms the other algorithms in terms of the total latency because the number of occupied cloudlets is smaller. What's more, although occupying fewer cloudlets as well, the proposed algorithm differs from the local search algorithm in that it has a more concentrated distribution of execution latencies, due to the fact that instead of jointly optimizing request assignment and computing resource allocation, the local search algorithm optimizes both in steps. From the above results and analysis, it can be concluded that the proposed algorithm not only has advantages in latency optimization but also effectively improves the utilization of computing resources, thus achieving better load balancing.

7 Conclusion

In this paper, we studied a joint optimization problem of request assignment and computing resource allocation for the multi-access edge computing network, aiming to minimize the total latency which includes VM instance boot times, request execution latencies, and transmission latencies. Since the formulated optimization problem is intractable, we decomposed it into two subproblems, which are the computing resource allocation subproblem and the request assignment subproblem. The Lagrangian multiplier method was utilized for addressing the resource allocation subproblem as it is convex, and an approximation algorithm was devised based on the primal-dual updating method for solving the request assignment subproblem. Moreover, the approximation ratio was also analyzed to verify the feasibility of the proposed algorithm. Experiment results illustrated that the proposed algorithm achieves better performance in terms of total latency, load balancing, and computational speed compared to the random, greedy, and local search algorithms.

The future work of the joint optimization of request assignment and computing resource allocation includes (1) considering the resource allocation of multiple types of VM instances and combining the resource allocation of VM instances to requests for further optimization of multi-request scenarios; (2) integrating communication, computing and storage resources as well as task offloading to achieve global task allocation optimization is a challenging topic.

References

[1] T. Taleb, K. Samdanis, B. Mada, H. Flinck, S. Dutta, and D. Sabella, "On multi-access edge computing: A survey of the emerging 5G network edge cloud architecture and orchestration," *IEEE Commun. Surv. Tut.*, vol. 19, no. 3, pp. 1657–1681, Jul.–Sep. 2017.

[2] Z. Ding, P. Fan, and H. V. Poor, "Impact of non-orthogonal multiple access on the offloading of mobile edge computing," *IEEE Trans. Commun.*, vol. 67, no. 1, pp. 375–390, Jan. 2019.

[3] H. A. Alameddine, S. Sharafeddine, S. Sebbah, S. Ayoubi, and C. Assi, "Dynamic task offloading and scheduling for low-latency IoT services in multi-access edge computing," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 3, pp. 668–682, Mar. 2019.

[4] Z. Tao *et al.*, "A survey of virtual machine management in edge computing," *Proc. IEEE Proc. IRE*, vol. 107, no. 8, pp. 1482–1499, Aug. 2019.

[5] L. Zhao and J. Liu, "Optimal placement of virtual machines for supporting multiple applications in mobile edge networks," *IEEE Trans. Veh. Technol.*, vol. 67, no. 7, pp. 6533–6545, Jul. 2018.

[6] M. Chen and Y. Hao, "Task offloading for mobile edge computing in software defined ultra-dense network," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 3, pp. 587–597, Mar. 2018.

[7] L. Li, K. Ota, and M. Dong, "DeepNFV: A lightweight framework for intelligent edge network functions virtualization," *IEEE Netw.*, vol. 33, no. 1, pp. 136–141, Jan./Feb. 2019.

[8] Z. Kuang, L. Li, J. Gao, L. Zhao, and A. Liu, "Partial offloading scheduling and power allocation for mobile edge computing systems," *IEEE Internet Things J.*, vol. 6, no. 4, pp. 6774–6785, Aug. 2019.

[9] Y. Ye, R. Q. Hu, G. Lu, and L. Shi, "Enhance latency-constrained computation in MEC networks using uplink NOMA," *IEEE Trans. Commun.*, vol. 68, no. 4, pp. 2409–2425, Apr. 2020.

[10] S. Tian, C. Chi, S. Long, S. Oh, Z. Li, and J. Long, "User preference-based hierarchical offloading for collaborative cloud-edge computing," *IEEE Trans. Services Comput.*, to be published, doi: 10.1109/TSC.2021.3128603.

[11] Z. Liang, Y. Liu, T.-M. Lok, and K. Huang, "Multi-cell mobile edge computing: Joint service migration and resource allocation," *IEEE Trans. Wireless Commun.*, vol. 20, no. 9, pp. 5898–5912, Sep. 2021.

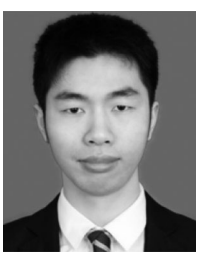
[12] T. X. Tran and D. Pompili, "Joint task offloading and resource allocation for multi-server mobile-edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 1, pp. 856–868, Jan. 2019.

[13] Y. Liao, L. Shou, Q. Yu, Q. Ai, and Q. Liu, "Joint offloading decision and resource allocation for mobile edge computing enabled networks," *Comput. Commun.*, vol. 154, pp. 361–369, 2020.

[14] J. Plachy, Z. Becvar, E. C. Strinati, and N. D. Pietro, "Dynamic allocation of computing and communication resources in multi-access edge computing for mobile users," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 2089–2106, Feb. 2021.

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 01:37:41 UTC from IEEE Xplore. Restrictions apply.

[15] T. G. Rodrigues, K. Suto, H. Nishiyama, N. Kato, and K. Temma, "Cloudlets activation scheme for scalable mobile edge computing with transmission power control and virtual machine migration," *IEEE Trans. Comput.*, vol. 67, no. 9, pp. 1287–1300, Sep. 2018.
[16] T. Bahreini, H. Badri, and D. Grosu, "Mechanisms for resource allocation and pricing in mobile edge computing systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 3, pp. 667–682, Mar. 2022.
[17] J. Li, W. Liang, and Y. Ma, "Robust service provisioning with service function chain requirements in mobile edge computing," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 2138–2153, Feb. 2021.
[18] Y. Liu, H. Lu, X. Li, Y. Zhang, L. Xi, and D. Zhao, "Dynamic service function chain orchestration for NFV/MEC-Enabled IoT networks: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 8, no. 9, pp. 7450–7465, May. 2021.
[19] M. Wang, B. Cheng, W. Feng, and J. Chen, "An efficient service function chain placement algorithm in a MEC-NFV environment," in *Proc. IEEE Glob. Commun. Conf.*, 2019, pp. 1–6.
[20] X. Wang *et al.*, *AI: Convergence of Edge Computing and Artificial Intelligence*. Berlin, Germany: Springer, 2020.
[21] T. Gao *et al.*, "Cost-efficient VNF placement and scheduling in public cloud networks," *IEEE Trans. Commun.*, vol. 68, no. 8, pp. 4946–4959, Aug. 2020.
[22] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint computation offloading and user association in multi-task mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 67, no. 12, pp. 12 313–12 325, Dec. 2018.
[23] T. X. Tran and D. Pompili, "Joint task offloading and resource allocation for multi-server mobile-edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 1, pp. 856–868, Jan. 2019.
[24] D. Molin, F. Achten, M. Bigot, A. Amezcua-Correa, and P. Sillard, "WideBand OM4 multi-mode fiber for next-generation 400 Gbps data communications," in *Proc. IEEE Eur. Conf. Opt. Commun.*, 2014, pp. 1–3.
[25] H.-C. An and O. Svensson, "Recent developments in approximation algorithms for facility location and clustering problems" in *Combinatorial Optimization and Graph Algorithms*. Berlin, Germany: Springer, 2017, pp. 1–19.
[26] K. Tammer, "The application of parametric optimization and imbedding to the foundation and realization of a generalized primal decomposition approach," *Math. Res.*, vol. 35, pp. 376–386, 1987.
[27] S. P. Boyd and L. Vandenberghe, *Convex Optimization*. New York, NY, USA: Cambridge Univ. Press, 2004.
[28] D. P. Williamson and D. B. Shmoys, *The Design of Approximation Algorithms*. New York, NY, USA: Cambridge Univ. Press, 2011.
[29] K. Jain, M. Mahdian, E. Markakis, A. Saberi, and V. V. Vazirani, "Greedy facility location algorithms analyzed using dual fitting with factor-revealing LP," *J. ACM*, vol. 50, no. 6, pp. 795–824, 2003.
[30] Y. Ma, W. Liang, M. Huang, W. Xu, and S. Guo, "Virtual network function service provisioning in MEC via trading off the usages between computing and communication resources," *IEEE Trans. Cloud Comput.*, to be published, doi: 10.1109/TCC.2020.3043313.
[31] A. Sheoran, X. Bu, L. Cao, P. Sharma, and S. Fahmy, "An empirical case for container-driven fine-grained VNF resource flexing," in *Proc. IEEE Conf. Netw. Function Virtualization Softw. Defined Netw.*, 2016, pp. 121–127.
[32] L. Li, T. Q. Quek, J. Ren, H. H. Yang, Z. Chen, and Y. Zhang, "An incentive-aware job offloading control framework for multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 20, no. 1, pp. 63–75, Jan. 2021.
[33] Y. Qu *et al.*, "Robust offloading scheduling for mobile edge computing," *IEEE Trans. Mobile Comput.*, to be published, doi: 10.1109/TMC.2020.3043000.
[34] IBM, IBM ILOG CPLEX optimizer, 2011. [Online]. Available: http://www-01.ibm.com/software/integration/optimization/cplex-optimizer/



Haolin liu received the BEng degree and the PhD degree from the Sichuan University, Chengdu, China, in 2010 and 2015, respectively. He is currently an associate professor with the Key Laboratory of Hunan Province for Internet of Things and Information Security and School of Computer Science, Xiangtan University, Xiangtan, China. His research interests include mobile edge computing, wireless sensor networks, and the Internet of Things.

Xiaoling Long received the BS degree in communication engineering from the Hunan University of Engineering, Xiangtan, China, in 2017. She graduated degree from Xiangtan University, Xiangtan, China, with the MS degree in information and communication engineering, in 2021. Her research interests include multi-access edge computing and network functions virtualization.

Zhetao Li (Member, IEEE) received the BEng degree from the Xiangtan University, Xiangtan, China, in 2002, the MEng degree from the Beihang University, Beijing, China, in 2005, and the PhD degree in computer application technology from the Hunan University, Changsha, China, in 2010. He is currently a professor with the Key Laboratory of Hunan Province for Internet of Things and Information Security and School of Computer Science, Xiangtan University. He was a visiting Researcher with Ajou University from May to August 2012. His research interests include cloud computing, artificial intelligence, and multimedia signal processing.

Saiqin Long received the BEng degree in software engineering from the Hunan Normal University and the PhD degree in computer applications technology with the South China University of Technology, in 2009, 2014, respectively. She was an associate professor in the School of Computer Science, Xiangtan University, in 2017. Her research interests include cloud computing, edge computing, parallel and distributed systems, file systems, computer system architecture. She is a member of Chinese Computer Federation (CCF).

Rong Ran (Member, IEEE) received the PhD degree from Yonsei University, Seoul, South Korea, in 2009. She joined the Electrical and Telecommunication Research Institute (ETRI), Daejeong, South Korea, in 2009 and performed research on the IEEE802.16 m standardization. She was a research associate with the Hong Kong University of Science and Technologies, Hong Kong, in 2010. She is currently an assistant professor with the Department of Electrical and Computer Engineering, Ajou University, Suwon, South Korea. Her current research interests include wireless communication and networks, sparse-signal processing, and Artificial Intelligence.

Hui-Ming Wang (Senior Member, IEEE) received the BS and PhD degrees in electrical engineering from the Xi'an Jiaotong University, Xi'an, China, in 2004 and 2010, respectively. From 2007 to 2008 and from 2009 to 2010, he was a visiting scholar with the Department of Electrical and Computer Engineering, University of Delaware, USA. He is currently a full professor with the Xi'an Jiaotong University. He has coauthored the book *Physical Layer Security in Random Cellular Networks* (Springer, 2016) and authored or coauthored over 150 IEEE journals and conference papers. His research interests include 5G communications and networks, intelligent communications, physicallayer security, and covert communications. He was the Clarivate Highly Cited Researcher, in 2019. He received the IEEE ComSoc Asia-Pacific Best Young Researcher Award in 2018, the National Excellent Doctoral Dissertation Award in China, in 2012, and the Best Paper Award from the IEEE/CIC International Conference on Communications in China in 2014. He is also an associate editor of the *IEEE Transactions on Communications*.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 01:37:41 UTC from IEEE Xplore. Restrictions apply.